HOLA: A Hostless Co-Simulation Architecture for Hardware-Accelerated CPU Verification

MICRO 2026 Submission **#13** – Confidential Draft – Do NOT Distribute!!

Abstract

Hardware-assisted simulation platforms such as emulators and FPGAs can run at tens to hundreds of MHz, but their effective verification throughput is bottlenecked by communication between hardware-simulated designs under test (DUTs) and host-executed reference models (REFs) during co-simulation. We observe that moving the entire verification framework onto hardware eliminates RTL-host communication and fully exploits hardware-assisted acceleration. To this end, we propose HOLA, a hostless co-simulation architecture that re-architects the REF as synthesizable hardware logic. Within this framework, RCore and RCache leverage DUT execution information to form a hinted REF microarchitecture that eliminates register dependencies and reduces memory traffic, enabling simple yet high-throughput ISA-compliant reference execution. To ensure correctness and debuggability of such synthesizable REFs at both construction time and runtime, HOLA provides a disciplined framework for ISA semantic realization, lifecycle-managed execution hints, and synthesizable software-like debugging primitives. A HOLA prototype supporting the RV64IMACB ISA achieves 60 MHz on FPGA, delivering a 7.7× speedup over the state-of-the-art RTL-host co-simulation framework for CPU verification.

Keywords

CPU Verification, Hostless Co-Simulation, Hardware-Accelerated Verification, Reference Model

1 Introduction

CPUs are growing increasingly complex, driven by sophisticated microarchitectures [62] and rapidly expanding ISAs. For example, the RISC-V RVA23 profile for application processors mandates 57 extensions and doubles the ISA specification length to 830 pages compared with 2019 [29, 30]. The increased complexity of designs and ISAs magnifies the burden of functional verification, which already dominates CPU development, often consuming over 60% of project time [41], yet only 14% of designs achieve first-silicon success, with 65% of respins caused by logic flaws [22].

Simulation-based verification [16, 32, 33, 60, 63], particularly co-simulation [14, 34, 36, 77], has been the standard CPU verification approach for decades. It simulates a register-transfer level (RTL) CPU design under test (DUT) against a simple, software-based reference model (REF), and verifies the DUT's correctness by comparing its instruction-level execution results to those of the REF.

As shown in Figure 1(a), common software-based practices rely on software-executed RTL simulators where the DUT is compiled into an executable binary. However, as circuit complexity grows, these simulators fail to scale, with modern out-of-order CPUs such as XiangShan [77] simulated at under 1 kHz. Despite extensive efforts to improve their performance [5, 6, 11, 20, 53, 73, 74, 80], RTL simulators rarely reach MHz-level speeds due to inherent dependencies and limited parallelism in circuit simulation.

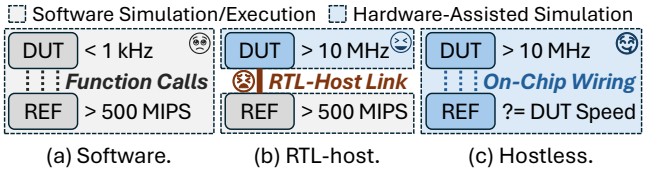


Figure 1: Co-simulation architectures for CPU verification.

To improve verification efficiency, hardware-assisted RTL simulation platforms, such as emulators [10, 17–19, 67, 70] based on application-specific integrated circuits and field programmable gate arrays (FPGAs) [12, 24, 35, 71, 72], have emerged as promising alternatives. As illustrated in Figure 1(b), these platforms accelerate co-simulation by mapping the DUT onto physical hardware, achieving simulation speeds ranging from several MHz to hundreds of MHz. However, software-based REFs cannot be deployed on hardware platforms. Instead, they still execute on a host machine (e.g., the PS on Zynq FPGAs), forming an RTL-host architecture for hardware-accelerated CPU verification [38, 45, 61, 66, 78, 79].

However, the efficiency of the RTL-host co-simulation is fundamentally limited by RTL-host communication overhead, and it becomes increasingly severe for modern CPUs with broad ISA verification requirements. For example, although Fromajo operates at over 1 MHz, it remains far below the native FireSim speed of hundreds of MHz for SonicBOOM [35, 79]. More generally, the RISC-V Verification Interface (RVVI) [59] specifies 145,637 bits of DUT output per cycle for full co-simulation of a 32-bit RISC-V CPU with floating-point, vector, and CSR features, implying an impractical 1,777.8 GB/s bandwidth without effective compression. Even after reducing communication overhead by 98.8%, DiffTest-H [78], the state-of-the-art RTL-host co-simulation framework, achieves only 7.8 MHz, still far below the 50 MHz DUT-only speed.

These observations expose a fundamental scalability bottleneck: as CPU and ISA scale, verification demands inevitably generate massive state traffic that overwhelms the bandwidth of any host-hardware link. Our key insight is that *this bottleneck arises from the split between DUT and REF across the RTL-host boundary*. If the REF can also be implemented on hardware, verification data exchange is done by on-chip wiring and operates at a native simulation speed.

To this end, this paper *proposes* HOLA, a HOstLess co-simulation Architecture that demonstrates for the first time that one can co-simulate the DUT and the REF entirely on hardware, enabling efficient hardware-accelerated CPU verification at near-native speeds. As illustrated in Figure 1(c), HOLA re-architects a CPU REF using synthesizable hardware logic and implements it alongside the

MICRO 2026, Athens, Greece
2026. ACM ISBN 978-X-XXXX-XXXX-X/XX/XX
https://doi.org/XXXXXXX.XXXXXXX

DUT on the same simulation platform, completely eliminating the communication overhead between the DUT and the REF.

Realizing this, however, introduces two main challenges.

(1) *Given the strict hardware implementation constraints, how can we design an efficient hardware REF that co-simulates with the DUT without incurring frequent stalls?* The co-simulation process compares execution results between the DUT and the REF, meaning overall progress is bounded by the slower of the two. However, unlike the DUT that leverages aggressive microarchitectural features such as out-of-order scheduling, the REF must remain structurally simple and reliable to preserve its role as a golden model. This creates a dilemma: the REF must achieve instruction throughput comparable to a high-performance DUT while retaining the simplicity essential for verification. To address this, we propose RCore and RCache that exploit DUT information as execution hints to eliminate register dependencies and reduce memory traffic, forming a simple yet efficient REF microarchitecture with full ISA compliance.

(2) *Given the design complexity of a hardware-based hostless co-simulation framework, how can we preserve its correctness at construction time and debuggability at runtime?* Software REFs benefit from high-level programming abstractions that precisely capture the ISA semantics required for correct reference execution. They also provide rich debugging primitives such as assertions and logs for observing execution. Even in RTL-host co-simulation, these capabilities remain available through the host that monitors DUT execution and assists diagnosis. In a hostless architecture, however, both construction-time correctness and runtime debuggability are harder to achieve, since the REF must be implemented entirely in synthesizable hardware descriptions without host-side software support. To this end, we propose the Hostless Diagnosis Framework (HDF), the correctness and debugging substrate for synthesizable REFs that supports ISA semantic realization, lifecycle-managed execution hints, and software-style debugging primitives.

We demonstrate the feasibility of HOLA with an open-source implementation that supports the RISC-V RV64IMACB instruction set across M, S, and U privilege modes. Experimental results show that HOLA achieves 1.17 MHz on an emulation platform when verifying XiangShan [48, 75], a competitive, six-issue, out-of-order CPU, corresponding to a 1.9× speedup over DiffTest-H, the state-of-the-art RTL-host co-simulation framework [78]. On an FPGA, HOLA reaches 60 MHz when verifying NutShell [49], a single-issue in-order CPU, delivering a 7.7× improvement over DiffTest-H.

This paper makes the following contributions:

- We propose HOLA, the first hostless co-simulation architecture that executes both DUT and REF entirely on hardware-assisted simulators at near-native CPU verification speeds.
- We propose RCore and RCache that leverage DUT information as execution hints to enable a simplified yet high-throughput hardware REF, and HDF, a diagnosis framework for preserving correctness and debuggability.
- We implement and open-source[1] a HOLA prototype for RISC-V CPUs and evaluate it on both FPGA and emulation platforms, demonstrating 1.9×–7.7× simulation performance improvements over prior RTL-host approaches.

[1]URL here. Disclosed upon paper acceptance.

2 Background

2.1 CPU Verification and Co-Simulation

CPU verification techniques are generally categorized into static and dynamic approaches. Static verification employs formal methods to pursue exhaustive correctness guarantees. However, these methods are hindered by the state space explosion problem, particularly in modern, large-scale designs [22]. In contrast, dynamic verification validates design correctness through simulation with specific inputs. While it cannot ensure full coverage, carefully constructed high-quality test cases can significantly improve coverage and efficiency. Due to its lower deployment cost, dynamic verification is widely adopted in industrial practice [15, 22, 63].

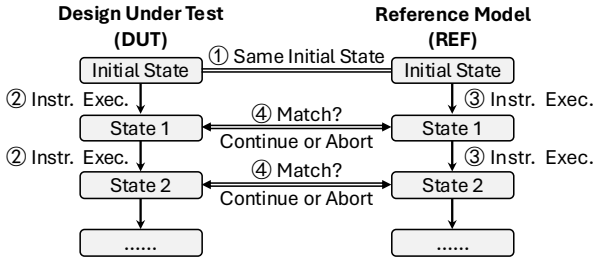


Figure 2: A typical co-simulation verification workflow.

Co-simulation, a dynamic verification method targeting ISA compliance, has emerged as the de facto CPU verification standard. As illustrated in Figure 2, a typical co-simulation workflow begins with ① synchronizing the ISA states between the DUT and a REF, such as the program counter (PC), register file, and memory contents. During RTL simulation, the DUT ② fetches and executes instructions. Upon each instruction commit, the co-simulation framework ③ notifies the REF to execute an instruction, and ④ compares the architectural states of the DUT and the REF. Co-simulation continues or aborts based on whether the results are consistent. Since the ISA provides a consistent abstraction across diverse microarchitectural implementations, co-simulation enables broad applicability and robustness in CPU verification [14, 31, 34, 77, 78].

During co-simulation, a REF provides the architecturally correct behavior against which the DUT is checked. In practice, REFs are most commonly implemented as instruction set simulators (ISSes) [34, 58, 77], because they offer modular instruction semantics, rich debuggability, and well-validated ISA-level behaviors.

Behavioral nondeterminism [14, 34, 38, 54, 76, 77] further complicates co-simulation, because an ISA-compliant ISS implementation may still realize only one concrete execution path even when the ISA permits multiple legal outcomes. As a result, a REF is not merely an arbitrary CPU or ISS implementation, but one adapted specifically for verification: it must accept a set of acceptable outcomes by refining its behavior on-the-fly rather than enforce a single deterministic path. For example, the RISC-V Linux kernel employs a lazy `sfence.vma` mechanism that allows page table entries (PTEs) to be allocated without immediate TLB flush. As a result, a subsequent memory access might either use the updated PTE, proceeding normally, or hit a stale TLB entry, triggering a page fault [77]. A REF used in co-simulation must therefore treat both behaviors as architecturally acceptable outcomes during checking.

The efficiency of co-simulation typically depends on the underlying RTL simulation speed (§2.2) and the additional overhead of DUT-REF coordination (§2.3). We next review prior work on them.

2.2 Accelerated RTL Simulation

Software-based register-transfer level (RTL) simulators generally adopt either an event-driven or a cycle-accurate execution paradigm. Among the two, the cycle-accurate approach offers greater potential for performance optimization and has seen increasing adoption in recent years [5, 6, 20, 73, 74, 82]. However, since software simulators inherently emulate circuit-level hardware behavior using general-purpose instructions, their performance degrades rapidly as chip design complexity grows. This scalability issue significantly limits the simulation efficiency, thereby motivating the adoption of hardware-assisted simulation techniques.

First, emulators use domain-specific chips and customized EDA tools that translate RTL designs into specialized circuit simulation instructions, exploiting both the extreme computational throughput of each chip and large-scale parallelism across multiple chips [17–19, 64]. Representative commercial products, such as Cadence Palladium, Synopsys ZeBu, and Siemens Veloce [10, 67, 70], achieve peak simulation speeds of up to 4 MHz. They provide orders-of-magnitude speedups over software RTL simulators while also offering mature support for hardware-software co-development.

Second, FPGA-based simulation synthesizes RTL directly onto programmable hardware, providing an execution mode closer to post-silicon environments and enabling simulation speeds above 100 MHz [12, 24, 35, 71, 72]. As a result, FPGAs have become a key platform for running lengthy full-system workloads. Compared with emulators, however, FPGA platforms are constrained to synthesizable hardware logic and limited per-device capacity, which introduces additional adaptation effort for verification [8, 40].

Yet hardware-assisted simulators still face a key limitation in CPU co-simulation: they achieve their highest efficiency only on synthesizable RTL. FPGA platforms require synthesizable designs directly, while emulators, although more flexible, also deliver their best performance on synthesizable RTL subsets. As a result, high-level CPU REFs, most commonly implemented in C++, cannot be integrated into the accelerated simulation flow at comparable speed and typically fall back to host-side execution.

2.3 RTL-Host Co-Simulation Architecture

Q9 (Rev-B)

Modern co-simulation follows an asymmetric producer-consumer model driven by DUT execution traces that contain nondeterministic guidance information. The REF can process these traces asynchronously and determine whether the DUT execution is correct. Because the REF is typically implemented in software, hardware-accelerated co-simulation commonly adopts an RTL-host architecture. In this model, the DUT runs on a hardware-assisted RTL simulator, while the software-based REF executes on a host machine (e.g., an x86 or ARM server, the PS side of Xilinx Zynq FPGAs). During co-simulation, the two sides exchange verification data, such as instruction commit events and architectural states, through high-speed links (e.g., PCIe, Ethernet, or InfiniBand). As a result, the overall RTL-host co-simulation performance is determined by three key factors: DUT simulation speed (1–200 MHz), REF execution throughput, and RTL-host communication overhead.

Since software REFs can already execute at hundreds of millions of instructions per second (MIPS) [77, 81], prior work has primarily focused on reducing communication overhead to achieve MHz/MIPS-level CPU verification. For example, ZP Cosim [45, 61] employs parallel data transfer and asynchronous communication on Zynq FPGAs, achieving up to 0.37 MIPS when verifying BlackParrot CPU using the Dromajo framework [34]. Similarly, Fromajo [79], another Dromajo-enhanced framework for the SonicBOOM CPU on the FireSim FPGA platform [35], reaches above 1 MHz.

The state-of-the-art RTL-host co-simulation framework, DiffTest-H [78], builds on the DiffTest information probes [77] and further reduces communication overhead through semantic-aware data compression, batching, and asynchronous data transfer. Although it cuts RTL-host communication overhead by 98.8%, it still achieves only 7.8 MHz, far below the 50 MHz DUT-only speed. This gap highlights the fundamental limitation of the RTL-host architecture: communication remains on the critical path of co-simulation.

3 Motivation and Challenges

The inevitable growth of communication overhead in the RTL-host architecture motivates us to rethink co-simulation toward a more scalable architecture. As CPUs become more complex, co-simulation must transfer increasingly rich verification states between the DUT and the host. For example, Imperas proposes the RISC-V Verification Interface (RVVI) [59] that consists of 16 mandatory and 5 optional fields. However, even after omitting mandatory floating-point, vector, and CSR related fields, as well as optional ones, the remaining interface still carries 1,189 bits. At a DUT simulation speed of 100 MHz, this necessitates a data transfer bandwidth of 14.5 GB/s per CPU core, nearly saturating the theoretical capacity of a PCIe 4.0 x8 link. Moreover, RVVI still does not cover important correctness checking requirements like store instructions and multi-core consistency, which are supported in other frameworks [34, 77]. Adding such information would only further increase communication volume and worsen RTL-host co-simulation performance.

To eliminate this bottleneck and fully exploit hardware-assisted acceleration, we propose HOLA, a hostless co-simulation architecture that implements the entire verification framework, most critically the REF, as synthesizable hardware logic. By relocating the REF from the software host to on-chip hardware, DUT-REF communication is reduced to simple intra-chip signal wiring, thereby removing the fundamental bottleneck of RTL-host interaction. This opens the door to near-native hardware-assisted co-simulation speed, but also introduces key design challenges, particularly in the design of synthesizable REFs (Table 1).

3.1 Simple yet Efficient REF Microarchitecture

A REF must remain simple to serve as a trustworthy verifier and efficient enough to avoid becoming the throughput bottleneck of co-simulation. In software, these goals often align because simplicity benefits both correctness and speed [7, 56, 77] (SW ISS in Table 1). In hardware, however, this alignment no longer holds: simple microarchitectures often sacrifice throughput, whereas high

Q11 (Rev-C)

Table 1: Tradeoffs of different REF realizations.

Type		Simple→Fast	Simple→Correct	Debuggability
SW	ISS	✓	✓	✓
HW	CPU-like	✗	●	●
HW	HOLA	✓	✓	✓

✓ supported; ● not naturally preserved; ✗ unavailable.

performance typically requires substantially greater design complexity. For example, although an in-order CPU is much simpler than an out-of-order design, it generally executes instructions more slowly. Thus, prior redundancy-based fault tolerance techniques, such as DIVA [3], usually use a lightweight checker only for validating simple computational results instead of full ISA correctness.

Q13 (Rev-E)

This simplicity-performance gap becomes most critical in hostless co-simulation, where the DUT and the REF execute instructions in parallel and overall throughput is determined by the slower side. Since the DUT may itself be a high-performance design, e.g., achieving instruction-per-cycle (IPC) values of up to 8 in an 8-way microarchitecture, the much simpler REF must still sustain comparable execution throughput to avoid throttling the entire co-simulation. As a result, hostless co-simulation sharpens the central dilemma of synthesizable REF design: *when realized in hardware, the REF must preserve the simplicity needed for trustworthy verification, yet remain efficient enough to keep up with a high-performance DUT.*

Under this constraint, conventional CPU microarchitectural techniques, such as branch prediction and out-of-order scheduling, cannot be directly adopted in the REF, since they undermine the simplicity required for correctness (HW CPU-like in Table 1). Instead, the microarchitecture must be reconsidered from first principles.

Q11 (Rev-C)

To redesign the REF without sacrificing simplicity, we next decompose its throughput problem according to the fundamental responsibilities. At its core, a CPU REF serves only to model architectural state transitions defined by the ISA. From this perspective, the throughput problem of a hardware REF reduces to two responsibilities: updating register state through instruction execution, and maintaining memory state through reads and writes. *Register state* requires faithfully realizing instruction semantics while sustaining sufficient execution throughput under a simple microarchitecture. *Memory state*, in contrast, requires high access throughput over a large address space: a single instruction may induce multiple memory operations, including instruction fetches, data accesses, and even page table walks. While such operations can be handled trivially in software through array-based accesses, supporting them efficiently in hardware is far more difficult.

This decomposition, in turn, motivates a hinted microarchitectural strategy for hardware REFs. HOLA exploits a key insight that distinguishes synthesizable REFs from conventional CPUs: whereas a CPU must execute every instruction independently, a hardware REF can leverage co-execution with the DUT to avoid resolving every execution problem from scratch. Because the DUT and the REF are expected to execute the same program, the REF can consume DUT outputs as execution hints to accelerate its own execution, while still preserving independent validation of architecturally visible behavior. This observation enables even a simple in-order microarchitecture with static pipelines to achieve substantial speedups. We detail the corresponding design considerations for register and memory states in §4.1 and §4.2, respectively.

3.2 Programming and Debugging Support

CPU verification frameworks have traditionally been developed in software or with non-synthesizable behavioral code (e.g., C++ or UVM APIs), which prevents them from fully exploiting the acceleration capabilities of hardware simulation platforms. In HOLA, by contrast, the framework must be implemented entirely with synthesizable constructs in hardware description languages (HDLs) such as Verilog, SystemVerilog, or Chisel. This shift, however, makes programming and debugging significantly more challenging than in conventional software-based REF development.

On one hand, construction-time correctness becomes harder to preserve. A software REF is typically realized as an ISS and benefits from modular code organization, standardized instruction semantics, and extensive validation by the open-source community. In contrast, building a synthesizable REF with verification-oriented correctness, especially to capture complex instruction semantics, lacks comparable support. Moreover, as we introduce hinted execution to improve REF throughput, correctness also depends on disciplined management of those execution hints. However, existing high-level HDLs [4, 9, 13, 39] mainly provide circuit- and microarchitecture-level abstractions, but offer little support for architecture-level realization. This creates a clear need for hardware-oriented abstractions that can systematically migrate ISA semantics into synthesizable hardware while ensuring correctness.

On the other hand, debugging is equally critical for CPU verification but far harder to replicate in a fully synthesizable setting. Software frameworks and software-based simulators provide rich debugging facilities, including standardized primitives (e.g., `assert`), compile-time instrumentation for error detection, detailed logs, and crash reports such as coredumps. While RTL-host co-simulation architectures can rely on the host for monitoring and diagnosis, the hostless environment lacks a programmable host, making these approaches inapplicable. As a result, a hostless REF must recover debuggability in synthesizable form without relying on host-side software, thereby bridging software-style debug practices with hardware deployment.

These two requirements motivate our Hostless Diagnosis Framework (HDF, §4.3), which provides construction-time correctness through hardware-oriented semantic realization and lifecycle management for execution hints, and runtime debuggability through synthesizable software-style observability primitives.

4 Design

The HOstLess co-simulation Architecture (HOLA) adopts a fully synthesizable CPU verification framework, enabling both the DUT and the REF to run on hardware-assisted simulation platforms at near-native speeds. As illustrated in Figure 3, HOLA consists of three key components:

Reference Core (RCore) (§4.1) contains the instruction execution pipelines of the REF. It uses a deeply pipelined, 24-stage design, each dedicated to one execution operation, and is expected not to

Q8 (Rev-B)

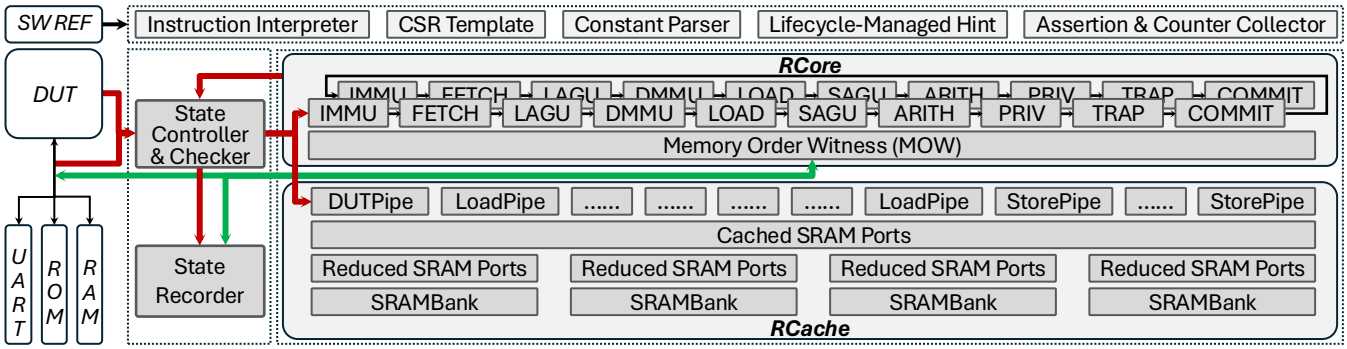


Figure 3: The HOstLess co-simulation Architecture (HOLA) framework for CPU verification.

become the critical path (§6.1). After each DUT instruction commit, the REF independently executes its corresponding instruction through the RCore pipelines, and the resulting architectural state is checked against that of the DUT. By consuming DUT information as execution hints, RCore preserves a simple in-order pipeline yet executes without hazards or stalls, sustaining high efficiency while faithfully modeling ISA-defined register-state semantics.

Reference Cache (RCache) (§4.2) is an optional component that manages all memory access requests from RCore when the RTL simulation platform cannot provide an ideal RAM (e.g., FPGA). Instead of issuing every request to external RAM, RCache uses limited on-chip storage to capture and reuse responses observed on the DUT's RAM interfaces. This design avoids duplicate RAM access operations and alleviates the pressure of maintaining memory state under high access demand in a large address space.

Hostless Diagnosis Framework (HDF) (§4.3) provides the correctness and debugging substrate for synthesizable REFs in hostless co-simulation. It enables hardware-oriented ISA semantic realization, lifecycle-managed execution hints, and synthesizable software-style debugging primitives. Together, these mechanisms bring software-like construction-time correctness and runtime debuggability into a fully synthesizable co-simulation framework.

4.1 Reference Core (RCore)

The main pipeline of the REF is designed with two key considerations: simplicity and efficiency. Inspired by the modular decomposition used in software ISSes, we decompose instruction execution into modular stages (e.g., fetch, decode, execute, commit), which enables a straightforward and reusable hardware pipeline. To sustain efficiency, we further leverage execution information (e.g., PC, architectural registers) from the DUT as REF hints, allowing the REF pipeline to run fully without hazards or stalls while still faithfully modeling instruction semantics for the register state.

On this basis, RCore implements a fully pipelined execution engine (§4.1.1), whose stages are illustrated in Figure 3. When verifying a superscalar CPU with an IPC greater than one, RCore chains multiple pipeline copies together, each responsible for verifying one committed instruction from the DUT. This design incurs little area (§6.1) but greatly improves efficiency. To preserve the program-order effects of memory operations, RCore integrates a Memory Order Witness (MOW) outside the pipeline stages, which routes and serializes memory requests from multiple pipeline copies (§4.1.2).

Q7 (Rev-B)

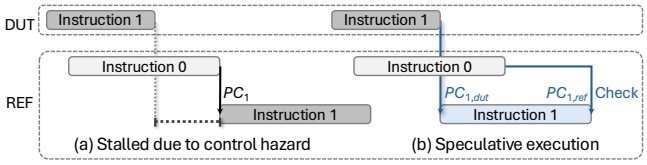


Figure 4: Control hazards in instruction executions.

4.1.1 Eliminating Hazards. In HOLA, when the DUT commits n instructions, RCore is notified to execute n instructions as well. These instructions sequentially pass through the pipeline stages and complete the corresponding operations at each stage, until the final commit stage where the REF results are compared against those from the DUT. Ideally, all instructions are expected to finish, whose results are then used for correctness comparison against those of the DUT, within a fixed time limit of $n \times k$ clock cycles, where k is the length of the execution pipeline.

However, instruction dependencies in the REF pipeline would normally cause frequent stalls, as later instructions rely on architectural states produced by earlier ones. Our key observation is that, in co-simulation, the DUT naturally exposes the same control-flow and state-update outcomes that the REF is expected to validate. RCore therefore uses such DUT-provided information as speculative execution hints, allowing dependent consumer instructions to proceed without waiting for earlier producers to complete.

To preserve correctness, however, these hints are used only to steer speculative execution and must be later validated through REF-side execution once the corresponding producer results become available. Incorrect DUT-provided hints therefore do not compromise REF independence; instead, they are exposed as mismatches during revalidation. This speculation-and-validation model forms the basis of RCore's hazard handling, while HDF (§4.3.2) further provides the programming primitives for managing the hints.

Control hazards. Instruction fetch depends on the updated program counter (PC) from the preceding instruction. As shown in Figure 4(a), without hinted speculation, REF must stall the fetch of Instruction 1 until its PC is computed. With DUT guidance, the

REF speculatively fetches the next PC ($PC_{1,dut}$) and later validates it against its own result ($PC_{1,ref}$), eliminating stalls (Figure 4(b)).

Data hazards. Later instructions may read registers or CSRs that are still being written by earlier instructions, e.g., an `add` consuming `RS1`/`RS2`, or an `sret` reading `sstatus` while a preceding `CSRW` is pending. In such cases, RCore uses DUT-provided architectural state as speculative operand hints to continue execution without waiting for the producer to commit in the REF pipeline. The corresponding values are then revalidated once the producer-side REF execution completes. If the DUT provides an incorrect state update, the mismatch is exposed during revalidation and flagged as a DUT bug, with no need for state rollback or recovery.

Structural hazards. Some arithmetic operations rely on long-latency, non-pipelined functional units, such as integer division, which would stall a simple pipeline if implemented directly. Rather than replicating such units in the REF, RCore validates DUT results using cheaper, semantically equivalent checking logic based on an inverse computation. For example, a division result can be checked by multiplying the quotient and divisor. This avoids costly replication of functional units while still preserving correctness.

4.1.2 Resolving Memory Orderings. While most register-side hazards can be resolved through speculative execution with DUT-provided hints, memory operations require separate handling because their correctness depends on the order in which load and store effects become visible over large shared memory state. To preserve this ordering without stalling every dependent access, HOLA introduces Memory Order Witness (MOW) as a centralized memory-side ordering mechanism. It allows loads to proceed speculatively using DUT-provided expected values, while buffering and delaying store effects until they reach the correct position in program order. In this way, RCore maintains efficient pipeline execution while preserving correct architectural memory behavior.

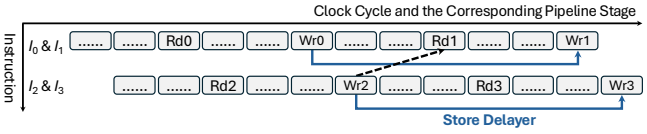


Figure 5: Write-After-Read (WAR) memory dependency.

Write-after-Read (WAR). As illustrated in Figure 5, a younger store (Wr2 of I_2) may be issued earlier than an older load (Rd1 of I_1), potentially overwriting the value before I_1 reads and observes its target value. To prevent this violation, the MOW buffers store requests from younger instructions in earlier pipelines and delays their architectural effect until the final store stage. This guarantees that all older loads (I_0, I_1) complete with the pre-store memory view before younger stores (I_2, I_3) are committed and become visible.

Read-after-Write (RAW). Delaying stores, however, introduces the dual problem shown in Figure 6. When an earlier store (Wr1 of I_1) is delayed, subsequent loads (Rd2 of I_2, Rd3 of I_3) may observe stale values before Wr1 takes effect. To preserve correctness without stalling these loads, the MOW attaches DUT-provided expected values as execution hints to the load requests and tracks the corresponding pending stores within a bounded window. Once the relevant store effects are resolved, the MOW revalidates that

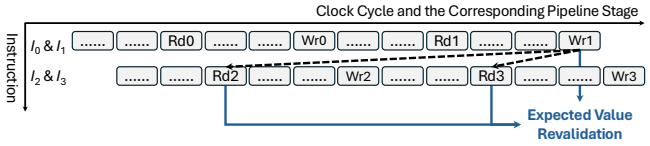


Figure 6: Read-After-Write (RAW) memory dependency.

the hinted, speculative load observations are consistent with the expected values under the finalized memory order.

By combining delayed-store serialization for WAR dependencies with expected-value revalidation for RAW dependencies, the MOW enforces correct memory ordering checking through a lightweight speculative validation mechanism. This enables RCore to execute memory operations efficiently without pipeline stalls.

4.2 Reference Cache (RCache)

Although RCore employs speculative execution to eliminate most pipeline hazards, it still requires a high-bandwidth, low-latency memory system that supports concurrent accesses from multiple pipelines. Specifically, each pipeline copy for single-instruction execution has 9 read ports (2 for cross-boundary instruction fetch, 1 for load, and 3/3 for instruction/data address translation) and 1 write port for store operations, all of which may issue requests concurrently. At the same time, these ports must provide responses within fixed latency to prevent pipeline stalls in RCore.

These stringent requirements bring structural hazards on platforms without an ideal main memory, such as FPGAs. Due to the limited bandwidth and uncertain latency of traditional memories, some requests may not be serviced in time. To alleviate this pressure, HOLA introduces RCache, an optional cache that reuses DUT-side memory interactions to avoid unnecessary accesses. It serves as a lightweight differential structure over DUT memory, preserving REF-visible stores while reusing DUT traffic whenever possible.

4.2.1 Management Policies. Traditionally, in co-simulation, the REF and the DUT maintain separate instances of main memory (RAM). Except for sharing the same initialization values, these RAMs are logically independent and physically isolated. Specifically, the dedicated REF RAM serves as the golden RAM reference and is never polluted by DUT behaviors.

To address the limitations of conventional REF RAM and support the high-bandwidth, low-latency demands of RCore, we propose RCache, an on-chip, small-size, DUT-assisted REF RAM. Instead of maintaining a complete and isolated RAM, RCache partially mirrors the DUT RAM and records only committed modifications, i.e., store effects that have become architecturally visible in the REF but have not yet been committed to the DUT RAM.

Given an initial RAM state R_0, after executing store instructions $s_0, ..., s_n$, the correct state of REF RAM R_{ref} is:

$$R_{ref} = R_0 \cup \{s_0, ..., s_n\}$$

Due to the presence of caches and other microarchitectural components, the DUT may not have written all stored data into RAM, resulting in the DUT RAM state R_{dut}:

$$R_{dut} \subseteq R_0 \cup \{s_0, ..., s_n\} = R_{ref}$$

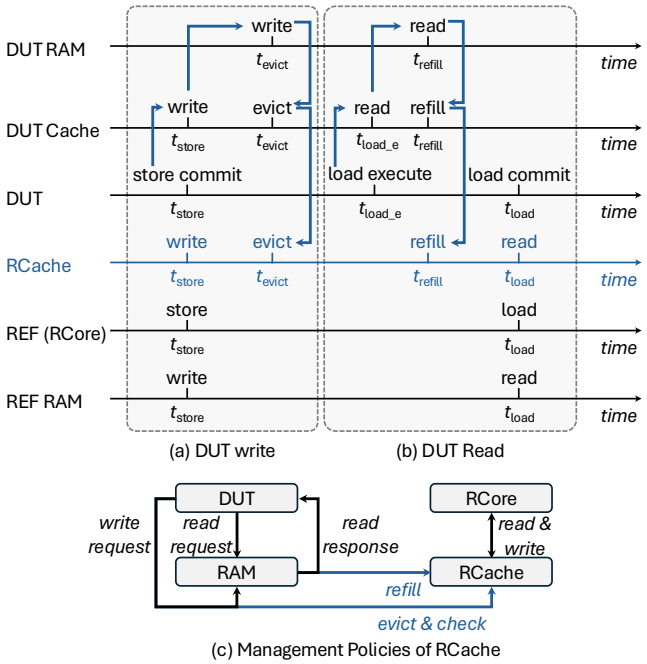


Figure 7: Examples of the RCache workflow.

Following this insight, we introduce RCache C_{ref}, which, in conjunction with R_{dut}, reconstructs R_{ref} as follows:

$$\begin{aligned} R_{ref} &= (R_{ref} \backslash R_{dut}) \cup R_{dut} \\ &= \{s_i | s_i \in R_{ref} \wedge s_i \notin R_{dut}\} \cup R_{dut} \\ &= C_{ref} \cup R_{dut} \end{aligned}$$

Conceptually, C_{ref} records only the committed memory effects that remain absent from the current DUT RAM view, while all other data are directly reused from DUT RAM. A concrete example for RCache is shown in Figure 7 with its effective timeline highlighted.

As shown in Figure 7(a), when the DUT executes a store at t_{store}, the REF executes the same instruction and writes the stored data into its RAM, making the corresponding store effect visible in its own memory view. Later, when the DUT flushes the same data into its RAM at t_{evict}, the corresponding memory contents in the REF and DUT views should become identical. Therefore, RCache only needs to preserve the stored data between t_{store} and t_{evict}: once stored data equivalence is confirmed at t_{evict}, the REF can safely discard this differential state and reuse data from DUT RAM.

Similarly, as shown in Figure 7(b), before committing a load instruction at t_{load}, the DUT fetches data from its RAM at t_{refill}. To avoid redundant accesses, RCache reuses this DUT-initiated RAM refill data to update itself, so that, at t_{load}, the same data can be returned as the response to RCore's read request. Importantly, RCache reuses the data returned by the golden DUT RAM, rather than an internal DUT state, so the reused value always corresponds to the correct RAM contents at the requested address.

In summary, RCache does not maintain a complete REF RAM. Instead, it records the committed differential write effects between REF RAM and DUT RAM, while reusing DUT-observed RAM refill and eviction traffic to mostly reconstruct the REF-visible memory state. As illustrated in Figure 7(c), RCache is therefore maintained passively under DUT guidance and actively by RCore stores.

4.2.2 RCache Misses. RCache assumes that an on-chip cache can retain enough differential write effects. When this assumption no longer holds under resource constraints, RCache misses may occur.

First, RCache may not be large (or smart) enough to retain all clean or reused blocks needed for REF execution.

- *Read Miss*: Accessing RCache may result in a cache miss, preventing the corresponding `read` from being verified. RCache may then optionally request a refill from DUT RAM to handle this with minor memory bandwidth overhead.
- *Evict Miss*: Due to differences in cache size and replacement policies between RCache and the DUT, when the DUT writes back a clean cache block to RAM, the corresponding block may already have been dropped from RCache, preventing that eviction from being validated. For DUTs that do not write clean cache blocks back to RAM, this case should not arise and is treated as a potential DUT bug.

Second, resource constraints on hardware-assisted simulation platforms may prevent implementation of some synthesizable logic, imposing additional design limitations on RCache. Specifically, RCache needs to handle multiple concurrent requests per clock cycle. Since these requests access the storage units that hold data in RCache, concurrent read and write operations to these units are required. However, on FPGAs, storage is typically implemented using Block RAM (BRAM) or UltraRAM (URAM), which generally support at most one read and one write request per clock cycle. Thus, RCache may not be able to service all read requests, although write requests must still be processed correctly to preserve committed differential state.

- *Port Miss*: Similar to a read miss, except that it occurs due to port arbitration limits for on-chip storage units. HOLA either leaves the request unchecked or defers it to a future idle port so that it can still be checked.

To avoid the miss events and improve access parallelism under FPGA constraints, RCache employs several lightweight microarchitectural techniques, including PLRU replacement, read-port caching/arbitration, write buffering, and storage banking. These techniques primarily reduce structural contention and improve RCache hit rate under limited on-chip storage bandwidth.

Overall, these miss events may leave some instructions' memory accesses or DUT clean evictions unchecked, or incur additional RAM bandwidth overhead if RCache performs refills on misses. Importantly, these miss events affect runtime REF checking coverage (§6.2), rather than the correctness of the REF itself. The DUT-provided execution hints are still subject to revalidation (§4.1, §4.3)

4.3 Hostless Diagnosis Framework (HDF)

A hostless co-simulation framework must be implemented entirely in synthesizable hardware, where development is substantially

Table 2: Programming and debugging support in HDF.

Feature	Software	HOLA
Instruction	Template	Operator Tree
CSR	`csr_t` class	`CSRDef` class
Constant	Macro / script	String interpolation
Hint	Ad hoc variables	Typed lifecycle management
Crash	Signal / coredump	Standalone mode
Assertion	`assert`	`assert`
Counter	`counter`	`counter`
Logging	`printf`, `display`	REF firmware

more difficult than in software. To address this challenge, we propose the Hostless Diagnosis Framework (HDF), which provides the correctness and debugging substrate for synthesizable REFs in hostless co-simulation, as listed in Table 2. HDF targets two complementary requirements: construction-time correctness and runtime debuggability. To this end, it adopts three mechanisms: (1) hardware-oriented ISA semantic realization, (2) lifecycle-managed execution hints, and (3) synthesizable debugging primitives.

4.3.1 ISA Semantic Realization. A central challenge in HDF is to migrate sophisticated ISA semantics from existing software REFs into synthesizable hardware without losing the structural clarity that makes REFs easy to develop and validate. In designs such as Spike, each *instruction* is implemented as an independent template (e.g., files under `riscv/insns`) that specifies its effects on the architectural state. However, such templates assume sequential execution, whereas hardware must encode all possible outcomes explicitly. For example, a software `if` executes only one branch, whereas hardware must represent both branches concurrently through conditional dataflow (e.g., `Mux`). Bridging this gap requires more than direct translation: it demands an intermediate abstraction that captures ISA state transitions in a form suitable for hardware construction.

HDF addresses this by parsing each template into an abstract syntax tree (AST) and transforming it into an operator tree, which serves as a hardware-oriented semantic representation. The key challenge is to reconcile the imperative, branch-oriented semantics of software templates with the concurrent, compositional nature of hardware. The operator tree bridges this gap by encoding ISA state transitions in a uniform set of nodes, enabling structured generation of synthesizable logic using Chisel. For RISC-V, we analyze Spike and distill 3 basic data types (`BValue`, `UValue`, `SValue`), 6 state transitions (`Assert`, `RedirectPC`, `XprWrite`, `Load`, `Store`, `AMO`), and 4 micro-operations (`StateTransition`, `NamedValue`, `When`, `Block`).

Beyond instructions, HDF treats CSRs and ISA-defined constants as part of the same semantic source that must be realized in hardware. *CSRs* introduce additional complexity because their semantics involve both explicit read/write accesses and implicit updates triggered by privilege events like exceptions and interrupts. In Spike, CSRs are implemented as subclasses of a `csr_t` base class, each overriding methods like `read`, `write`, and `verify_permissions`. HDF retains this modular view by introducing a hardware-side `CSRDef` base class. Developers implement CSR semantics once, while HDF uses Scala reflection to automatically identify the required CSRs under a given CPU configuration and generate the corresponding hardware descriptions. Besides, HDF integrates *constants* through a string-interpolation mechanism, translating symbolic names (e.g., `u"MATCH_ADD"`) directly into Chisel constants. Overall, HDF preserves the concise, modular coding style of software REFs while preserving the correctness of the constructed REF.

4.3.2 Lifecycle-Managed Execution Hints. Execution hints are central to hostless co-simulation because they allow the REF to speculatively advance using DUT-provided outcomes without stalls. However, these hints must be managed under an explicit lifecycle so that hinted execution remains correct and debuggable.

HDF treats execution hints as typed Scala/Chisel objects. A typed hint may carry, for example, a speculative next PC, an architectural register or CSR value, or an expected memory value. Each hint is explicitly associated with the instruction that produces and consumes it and implicitly managed by HDF. This explicit typing allows the generated REF logic to distinguish between ordinary REF-computed state and DUT-derived speculative hints.

The lifecycle of a typed hint consists of four steps. First, a hint is *defined* at an architectural interface where DUT-visible information becomes available. Second, it is *propagated* to the appropriate REF component, such as a pipeline stage in RCore. Third, it may be *consumed* to steer speculative execution, for example by enabling an instruction fetch to proceed before the REF computes the PC. Finally, it is *revalidated and retired*: once the corresponding REF-side producer result or memory ordering condition becomes available, the hint is checked against REF-side semantics.

This lifecycle discipline is essential for construction-time correctness and also supports runtime diagnosis. On one hand, hint consumers are embedded into the semantic interfaces generated from operator trees and CSR definitions, so hinted execution remains managed. On the other hand, hint mismatches or unvalidated hints become observable through HDF's debugging primitives at runtime, allowing developers to diagnose not only DUT bugs but also mistakes in the synthesizable REF implementation.

4.3.3 Debugging Support. Although HDF eliminates reliance on an external software host, effective debugging remains indispensable during CPU verification. To this end, HDF equips the REF with two complementary modes of operation. In *verification mode* (green lines in Figure 3), the REF co-simulates with the DUT and continuously checks for mismatches or assertion violations. Assertions and counters written in Chisel are automatically converted, via a customized FIRRTL transform, into synthesizable hardware checks and named performance metrics for real-time monitoring during simulation. Once an error is detected, the system transitions into *standalone mode* (red lines in Figure 3), where the REF operates as a self-contained, non-speculative CPU running dedicated debugging firmware. In this mode, recorded failure states are preserved for post-error analysis, and the firmware interprets them to emit structured logs while allowing developers to inspect execution states through standard I/O peripherals. This effectively embeds a debugging host into the hardware itself, restoring software-like diagnosis capabilities without relying on a host-side software process.

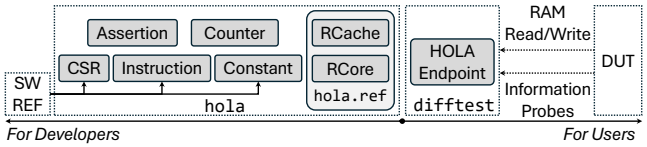


Figure 8: HOLA toolchain for CPU verification.

5 Implementation

We implement and open-source a prototype of HOLA based on the DiffTest [77] co-simulation framework. As illustrated in Figure 8, HOLA reuses the existing DiffTest interfaces and preserves a uniform co-simulation workflow for end users, while replacing the conventional software-hosted REF backend with a hardware-assisted hostless one for accelerated CPU verification.

For end users, the HOLA toolchain is fully compatible with the information probes defined in DiffTest, such as instruction commits and architectural register states. It naturally supports out-of-order cores by verifying the in-order committed instructions, independently of the DUT's internal execution order. To support RCache (§4.2.1), HOLA further extends these probes with optional memory-access information. We provide a lightweight Scala trait for memory read and write responses, that users only need to inherit from (or mix into) the DUT design and specify the top-level module explicitly. The toolchain then automatically discovers the DUT-RAM interfaces without requiring additional manual integration effort.

Q1 (Rev-A/C/E)

For framework developers, flexibility and extensibility are particularly important because synthesizable hardware construction is substantially more complex than software implementation. Accordingly, the HOLA implementation is organized around two categories of components. First, RCore and RCache together constitute the golden REF: RCore handles register-side ISA semantics and state updates, while RCache reconstructs and checks memory-side behavior during co-simulation. Second, HDF serves as the supporting infrastructure, providing correctness-preserving and software-like debuggability programming primitives for the synthesizable framework. With this organization, HOLA already supports the RISC-V RV64IMACB ISA with M, S, and U privilege modes, and also provides a scalable foundation for future ISA extensions. It is worth noting that current prototype already includes the floating-point register files for resource evaluation (§6.1), although the corresponding instruction operations are currently skipped.

Q2 (Rev-A/E)

Extending HOLA to support new instruction sets is straightforward. For lightweight extensions without new architectural states (e.g., bitmanip), developers only need to provide the instruction list, from which HDF automatically generates the corresponding semantics and hardware implementations. For more complex extensions that introduce additional states or privileged operations (e.g., floating-point units and CSRs in the F/D extensions), developers can extend the `ArchState` definition and add new operator nodes or CSRs through standardized interfaces. Relatively difficult, long-latency, non-pipelined arithmetic operations can be implemented either using cheaper validation logic such as inverse-result checking (e.g., DIV in §4.1.1) or using the same non-pipelined logic that may introduce stalls and performance overhead (e.g., FDIV). This modular design allows both simple and complex ISA extensions to be incorporated without disrupting the overall framework.

Q2 (Rev-A/E)

Table 3: Evaluation setup.

Feature	Configuration
DUT	DUT_s: NutShell, scalar, in-order, 128KB LLC DUT_m: XiangShan, MinimalConfig, 2-wide, out-of-order, 512KB LLC DUT_l: XiangShan, DefaultConfig, 6-wide, out-of-order, 2MB LLC
Platform	Emulator: Cadence Palladium FPGA: Xilinx xczu19eg-ffvc1760-2-i
Workload	Linux boot and `/init` run

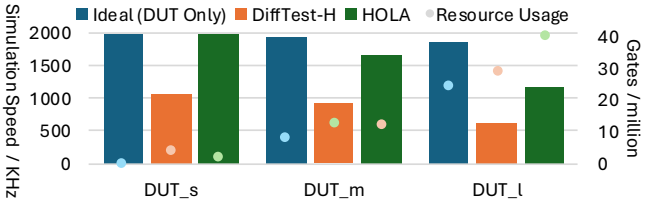


Figure 9: Simulation speed and resource usage on Palladium.

6 Evaluation

To evaluate HOLA in terms of verification efficiency, implementation cost, and resource trade-offs, we apply the HOLA toolchain to verify RISC-V processors of varying design complexity, including the in-order, scalar NutShell [49] and the 6-way superscalar, out-of-order XiangShan [48, 75], as shown in Table 3.

The evaluation is conducted on both the emulator and FPGA platforms, using the Linux kernel boot with the `/init` process as a throughput-oriented full-system workload. This workload generates a moderate volume of verification data, serving as a representative benchmark for the performance overhead of RTL-host co-simulation. Additionally, it exercises complex architectural and microarchitectural behaviors across all three privilege levels, with diverse memory access patterns for instructions, data, and page tables. These make it well-suited for evaluating the correctness and effectiveness of HOLA.

6.1 Verification Efficiency

The key advantage of HOLA is to leverage synthesizable hardware logic for hostless hardware-accelerated co-simulation. To assess this benefit, we compare its simulation speed against DiffTest-H [47, 78], the state-of-the-art RTL-host co-simulation framework.

6.1.1 Emulator. On Palladium that supports ideal main memories, as shown in Figure 9, we evaluate the verification speed and resource usage of HOLA with RCache disabled.

Our evaluation reveals that, while DiffTest-H achieves speeds of 0.6~1.1 MHz with RTL-host communication overhead significantly reduced by up to 99.8%, HOLA reaches 2.0 MHz, 1.7 MHz, and 1.2 MHz for DUTs of different scales, delivering speedups of 1.85×,

1.82×, and 1.88×, respectively. Compared with the ideal, native performance (i.e., simulation speed with only the DUT), HOLA attains 100%, 86%, and 63% of the maximum speed, reaching the platform's optimal performance for NutShell. We observe that, once any RTL-host interaction is present, the achievable speed is capped by an inherent ~1 MHz penalty, which limits the headroom of DiffTest-H even after aggressive communication reduction. HOLA removes this host-link bottleneck and therefore approaches DUT-only speed more closely, although the gap grows for wider DUTs as the synthesized REF itself scales with commit width.

Light-colored dots in Figure 9 show the gate usages. While HOLA consumes fewer gates and significantly outperforms DiffTest-H for small designs, as the DUT's superscalar width increases, the RCore pipeline length and its resource overhead grows proportionally.

6.1.2 FPGA. To overcome memory constraints and limited support for multi-ported storage, we enable RCache to alleviate memory access pressure and evaluate HOLA using DUT_s on a Xilinx zu19eg FPGA[2]. The results show that with a 512KB RCache configured as 8-way set-associative, 4-bank, and 3-read (implemented via 3 copies of the SRAMs), the HOLA framework, including the DUT, achieves an operating frequency of 60 MHz without timing violations. This indicates a superior 7.69× speedup over DiffTest-H (7.8 MHz).

This deployed point reflects a practical throughput-coverage trade-off. As discussed in §4.2.2, FPGA physical constraints may cause RCache misses. Under the deployed 512KB configuration with refill-on-read-miss enabled, HOLA prioritizes a timing-clean implementation, checks 99.93% of dynamic instructions, and validates 99.58% of DUT eviction requests, while incurring only 0.41% additional RAM bandwidth. The remaining instruction-side gap comes from residual Port Misses in RCache. If these residual misses are recovered by backpressure and clock gating for DUT, all dynamic instructions can still be checked, with an expected co-simulation throughput reduction below 1%, measured by multiplying the residual miss-event rate by the bounded response latency. The remaining eviction misses are specific to DUTs such as NutShell that may write back clean cache blocks to RAM and will not occur for the high-performance XiangShan CPU.

Q11 (Rev-C)

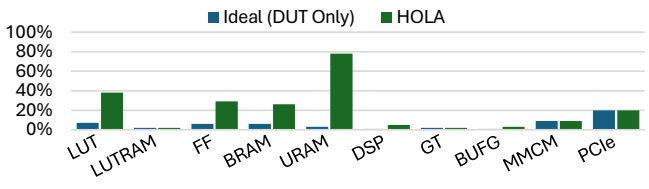


Figure 10: Resource usages on FPGA.

Figure 10 illustrates resource usage of HOLA on FPGA, showing notable overhead in LUT, FF, BRAM, URAM, and DSP. This motivates future optimizations including reducing FF and DSP usage for RCore and LUT, BRAM, and URAM usage for RCache. Timing reports indicate that the critical paths do not lie within HOLA.

Q8 (Rev-B)

To demonstrate the effectiveness of HOLA on realistic out-of-order designs, we synthesize DUT_m on the Xilinx VU19P FPGA.

Q1 (Rev-A/C/E)

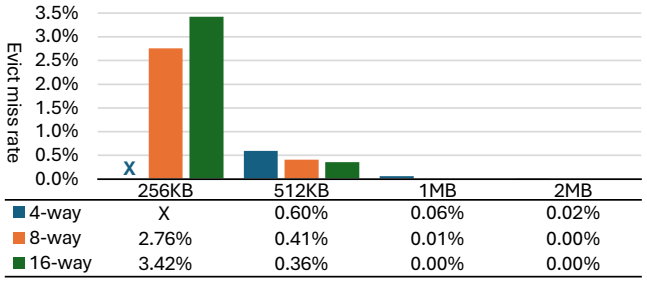
[2]This is one of the FPGA boards officially supported by the NutShell project. DUT_m and DUT_l cannot be accommodated due to FPGA constraints (limited LUTs).

DUT_m alone uses 14% LUTs and 20% BRAMs and reaches 50MHz. Integrating HOLA adds 31% LUTs (2% for RCore and 29% for RCache) and 43% BRAMs (for RCache), while preserving the same 50MHz frequency without timing violations. The evaluated RCache is configured as a 1 MB, 16-way, four-bank design with three read ports.

6.2 Effectiveness of RCache Trade-offs

The mechanisms in HOLA are complementary and not fully separable. Specifically, matching RCore width to DUT commit width is the natural design point (§4.1), whereas RCache has various configurations that impact miss rates and memory bandwidth overhead. This section evaluates its effectiveness of leveraging DUT RAM accesses and quantifies its resource trade-offs.

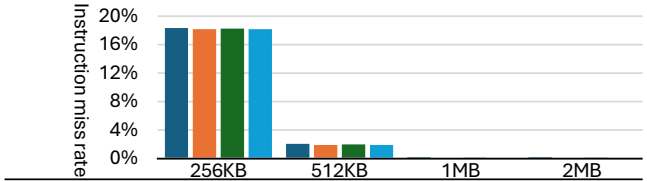
Q7 (Rev-B)



	256KB	512KB	1MB	2MB
4-way	X	0.60%	0.06%	0.02%
8-way	2.76%	0.41%	0.01%	0.00%
16-way	3.42%	0.36%	0.00%	0.00%

Figure 11: Evict miss rate across various configurations.

Evict Miss. RCache may evict a clean cache block earlier than the DUT, leading to a miss when the DUT later writes back that block but RCache fails to check its correctness. Figure 11 shows the evict miss rates across different RCache configurations[3]. Overall, the miss rate remains low and drops significantly with larger cache sizes or improved replacement strategies. For example, in the 256KB 8-way configuration, the miss rate is 2.76%, which can be further reduced to 0.53%, an 81% decrease, by adopting a PLRU policy.

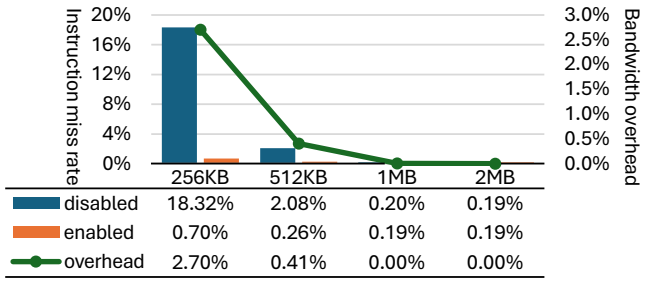


	256KB	512KB	1MB	2MB
2-bank, 2-port	18.32%	2.08%	0.20%	0.19%
2-bank, 3-port	18.18%	1.89%	0.02%	0.00%
4-bank, 2-port	18.27%	2.00%	0.12%	0.11%
4-bank, 3-port	18.18%	1.89%	0.01%	0.00%

Figure 12: Instruction miss rate across various configurations.

Read/Port Misses. Instruction misses occur when RCore accesses a block that does not exist in RCache (Read Miss) or when read requests fail due to port contention (Port Miss). Figure 12 shows that smaller caches suffer higher miss rates (e.g., 18% for 256KB), while increasing cache size and adding read ports significantly reduce misses (e.g., 1.89% for 512KB with three read ports).

[3]HOLA aborts in the 256KB 4-way case due to all blocks being dirty and non-replaceable.



	256KB	512KB	1MB	2MB
disabled	18.32%	2.08%	0.20%	0.19%
enabled	0.70%	0.26%	0.19%	0.19%
overhead	2.70%	0.41%	0.00%	0.00%

Figure 13: Impact of refill-on-read-miss on instruction miss rate and bandwidth overhead across various cache sizes.

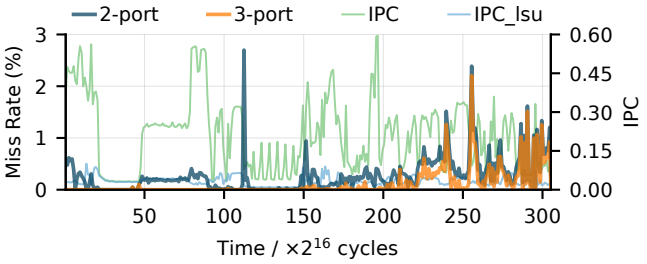


Figure 14: Phased instruction miss rate and instructions per cycle (IPC) during Linux boot. lsu: load/store instructions.

Bandwidth Overhead. RCache incorporates a refill mechanism that reloads data upon read misses to avoid unchecked requests. As shown in Figure 13, this mechanism significantly reduces miss rates, particularly for smaller caches, while incurring minimal bandwidth overhead. For instance, a 96.2%/87.5% miss rate reduction for a 256KB/512KB 2-port cache brings only 2.70%/0.41% overhead.

Q10 (Rev-C) To demonstrate the variation of RCache efficiency with workload memory pressure, we conduct a fine-grained phase analysis during the Linux boot process, which contains diverse execution behaviors with substantially different IPC, issue activity, and memory behavior. As shown in Figure 14, the two-port RCache exhibits larger miss spikes during the early, high-IPC but relatively regular kernel-initialization stages, primarily because of port contention. Adding a third port removes most of these misses. In contrast, the later Linux service-startup stages exhibit more irregular execution and memory activity. During these stages, the misses are less strongly correlated with IPC and mainly reflect differences between the simpler RCache policy and the more advanced DUT cache.

In summary, RCache keeps instruction miss rates low at modest hardware and bandwidth cost. With configurable optimizations such as replacement policies, banking, multi-porting, refill-on-miss, and optional backpressure on residual Read/Port Misses, RCache preserves instruction checking coverage at near-native speeds.

6.3 Hardware Programming Productivity

HDF plays a key role in the HOLA toolchain by enabling low-cost, automated migration of ISA semantics and debugging primitives. To evaluate its impact on hardware programming productivity, we perform a line-of-code analysis as a lightweight evaluation proxy.

Table 4: Lines of Chisel code (LoC) for modules in RCore.

Module	LoC	Class Definition
MMU	253	`MMU`, `IMMU`, `DMMU`, `MMUExpect`, `Sv39Trans`, `AddrTransInfo`, `PTEInterpreter`
FETCH	61	`FETCH`
AGU	52	`AGU`, `LAGU`, `SAGU`
LSU	126	`LSUOpcode`, `LSU`, `LoadUnit`, `StoreUnit`
ARITH	87	`Arithmetic`
PRIV	154	`Privileged`
TRAP	49	`TRAP`
COMMIT	25	`COMMIT`
RCore	129	`PipelineStage`, `PipelineConnect`, `RCore`

As summarized in Table 4, the complete implementation of RCore, including all pipeline stages for instruction execution, requires only 936 lines of Chisel code, which elaborate into 55,993 lines of SystemVerilog (60×). By comparison, we evaluate the XiangShan CPU and observe an elaboration ratio of 12×. This contrast demonstrates the effectiveness of HDF in reducing development complexity and supporting the efficient construction of synthesized REFs.

6.4 Finding Bugs

Like DiffTest-H and other ISA-level CPU co-simulation frameworks, HOLA detects bugs that eventually manifest as architectural-state mismatches. A key advantage of HOLA is to sustain long-running, full-program executions at near-native hardware-accelerated simulation speeds. Traditional verification often relies on short-running stimuli, such as handcrafted test cases, constrained random instruction sequences, or checkpoint-based tests [34, 65, 77]. They are effective for covering functional corner cases but fall short in exposing bugs that only emerge after prolonged execution or under realistic runtime conditions. HOLA overcomes this limitation by enabling whole programs to run to completion within practical time. In our evaluation, XiangShan passes all checkpoint-based tests from SPEC CPU2006/2017, yet HOLA reveals a hidden bug in `mstatus` CSR after 398.6M cycles of the full `416.gamess` benchmark, completing in only 6 minutes on the emulator. Though a short directed test may be constructed to reproduce this bug, it is still hard to reach by short tests before its trigger condition is known. This is because constructing directed tests requires deep expertise in both the DUT and the ISA. Instead, HOLA enables existing long-running workloads to be continuously checked at high speed, allowing rare architecturally visible bugs to surface without domain expertise.

Q5 (Rev-A/B)

Q5 (Rev-A/B)

7 Discussion

In this section, we discuss the limitations of the current HOLA prototype with future research directions in this field.

Larger designs under test. FPGA-based accelerated simulation inevitably faces the limited on-board resources. For DUT_l, the current limitation is FPGA capacity. DUT_l alone consumes 43% LUTs, 50% BRAMs, and 16% URAMs on VU19P FPGA. This leaves insufficient headroom for HOLA. A decoupled or partitioned multi-FPGA implementation [35, 72] is a well-established engineering approach to addressing this limitation. The HOLA architecture

Q1 (Rev-A/C/E)

also extends to multicore checking, which requires only additional observation of inter-core memory interleavings [34, 77], by adding an inter-core memory-update interface to RCache.

Non-ISA-level verification. HOLA targets system-level co-simulation based on ISA-level checking, which is widely used in chip verification and complements other techniques at different levels, such as unit testing, integration testing, and block testing. In this context, HOLA specifically addresses the efficiency bottleneck of co-simulation. While RCore and RCache address the key architectural challenges of system-level REFs, HDF can also detect non-ISA-visible bugs if designers expose the relevant invariants through assertions and counters.

Q2 (Rev-A/E), Q5 (Rev-A/B)

Verifying the verifier. A REF cannot be trusted merely because it is written in software or hardware. Instead, it can only be validated against another reference, whether an executable REF or the architectural behavior defined by the ISA and understood by domain experts. In current verification practice, established REFs (e.g., ISSes) become trusted through a combination of simple architectural semantics, modular instruction definitions, and long-term cross-validation against real CPUs and other simulators. Similarly, HOLA can itself be verified against existing REFs. When treated as a DUT, HOLA takes standard DiffTest verification traces as inputs and exposes output DiffTest interfaces. These inputs and outputs allow HOLA to be tested against existing REFs with general verification methodologies to verify that correct inputs produce REF/DUT consistency and that incorrect inputs are identified as mismatches.

Q6 (Rev-B)

8 Related Work

HOLA, to the best of our knowledge, is the first work to realize a fully functional REF in synthesizable hardware for hostless co-simulation, thus enabling accelerated CPU verification entirely on hardware-assisted simulation platforms. Nevertheless, prior studies have explored related challenges from different perspectives.

RTL-host architecture for accelerated co-simulation. A growing body of prior work has explored accelerated co-simulation using the RTL-host architecture [38, 45, 61, 66, 79]. In this setup, the DUT executes on the emulator/FPGA, while a software-based REF runs on an external x86 or ARM host, with verification data continuously exchanged between them. As CPU microarchitectures become more sophisticated and ISA extensions proliferate [29, 30], the amount of architectural state that must be checked and communicated during co-simulation steadily increases. This trend amplifies the cost of RTL-host communication, which already dominates the overall runtime. The state-of-the-art work, DiffTest-H [47, 78], reduces traffic by over 99.8% through carefully optimized data probes and reports 151 bugs, but the RTL-host communication remains the critical performance bottleneck for modern out-of-order CPUs. This widening gap between design complexity and communication efficiency motivates a necessary, fundamental paradigm shift toward the hostless co-simulation adopted by HOLA, where the REF is synthesized onto hardware and runs alongside the DUT.

Dynamic verification for post-silicon validation. As chip complexity increases and technology nodes shrink, researchers explore dynamic, in-situ techniques for detecting transient faults during post-silicon operation. DIVA [2, 3] introduces a lightweight instruction-level checker that validated execution results against unverified operands and opcodes, but its coverage was limited to data integrity. Argus [42] broadens this approach by combining four complementary checkers, covering control flow, dataflow, computation, and memory, thus providing more comprehensive fault detection across the pipeline. Redundant Multithreading [44, 55] takes a more heavyweight approach by executing two identical program streams in parallel to detect mismatches, but at the cost of duplicating hardware resources and significant performance overhead. While effective for exposing transient faults in deployed systems, these techniques target fault tolerance rather than functional verification. In contrast, HOLA revisits the simple on-chip checker paradigm pioneered by DIVA and extends the hardware architecture to address pre-silicon verification by providing a synthesizable REF that models complete ISA behaviors of CPUs.

Q13 (Rev-E)

Synthesizing ISA semantics. Prior work has explored various ways of synthesizing or abstracting ISA-level behaviors for hardware modeling and verification. Instruction-level abstraction (ILA) [27, 68, 69] introduces a unified formalism for capturing transactional behaviors of CPUs and accelerators. Formal ISA specifications such as Sail [1, 57] provide machine-readable descriptions of complete instruction sets. Meanwhile, informal implementations such as Spike [58] serve as widely used software references. High-level synthesis (HLS) [26, 28, 43, 52] raises the abstraction level of hardware design, but it sacrifices the fine-grained control required for constructing cycle-accurate REFs. In contrast, HOLA does not attempt to introduce a new formal ISA abstraction. Instead, it reuses existing software REFs such as Spike and migrates their semantics into synthesizable RTL. By preserving a modular code structure, HDF not only avoids redundant manual re-modeling of ISA semantics, but also provides developers with a unified, software-like environment for extending and debugging the framework.

Collaborative microarchitectures. Prior work has long leveraged useful, sometimes speculative, information from one architectural component for boosting the performance of another [21, 23, 25, 37, 46, 50, 51]. While such ideas are well established in CPU design, applying them to hardware REFs is nontrivial, since functional verification values correctness above performance. HOLA systematically analyzes the requirements of a CPU REF and, from this basis, introduces the RCore and RCache microarchitectures with carefully partitioned pipelines and DUT-REF memory interfaces, as well as the HDF for lifecycle-managed execution hints, thereby enabling the first hostless co-simulation framework.

9 Conclusion

We recognize that hostless co-simulation is promising for efficient CPU verification on hardware-assisted simulation platforms but creates new verification-oriented microarchitecture challenges. To address them, we present HOLA, a hostless co-simulation architecture that constructs the reference model entirely in synthesizable hardware. HOLA combines RCore for hazard-free register-side checking, RCache for efficient memory-side state reconstruction, and HDF for correctness-preserving REF construction and runtime debuggability. Our implementation achieves up to 1.9× and 7.7× speedup over the state-of-the-art on an emulator and FPGA, respectively, demonstrating its effectiveness in scaling CPU verification.

Rev-A/D

References

[1] Alasdair Armstrong, Thomas Bauereiss, Brian Campbell, Alastair Reid, Kathryn E. Gray, Robert M. Norton, Prashanth Mundkur, Mark Wassell, Jon French, Christopher Pulte, Shaked Flur, Ian Stark, Neel Krishnaswami, and Peter Sewell. 2019. ISA semantics for ARMv8-a, RISC-v, and CHERI-MIPS. *Proc. ACM Program. Lang.* 3, POPL, Article 71 (Jan. 2019), 31 pages. https://doi.org/10.1145/3290384

[2] Todd Austin, Valeria Bertacco, David Blaauw, and Trevor Mudge. 2005. Opportunities and challenges for better than worst-case design. In *Proceedings of the 2005 Asia and South Pacific Design Automation Conference*. 2–7.

[3] Todd M Austin. 1999. DIVA: A reliable substrate for deep submicron microarchitecture design. In *MICRO-32. Proceedings of the 32nd Annual ACM/IEEE International Symposium on Microarchitecture*. IEEE, 196–207.

[4] Jonathan Bachrach, Huy Vo, Brian Richards, Yunsup Lee, Andrew Waterman, Rimas Avižienis, John Wawrzynek, and Krste Asanović. 2012. Chisel: constructing hardware in a Scala embedded language. In *Proceedings of the 49th Annual Design Automation Conference* (San Francisco, California) *(DAC '12)*. Association for Computing Machinery, New York, NY, USA, 1216–1225. https://doi.org/10.1145/2228360.2228584

[5] Scott Beamer. 2020. A case for accelerating software RTL simulation. *IEEE Micro* 40, 4 (2020), 112–119.

[6] Scott Beamer, Thomas Nijssen, Krishna Pandian, and Kyle Zhang. 2021. ESSENT: A high-performance RTL simulator. In *Workshop on Open-Source EDA Technology (WOSET), at International Conference on Computer-Aided Design (ICCAD)*.

[7] Fabrice Bellard. 2005. QEMU, a fast and portable dynamic translator.. In *USENIX annual technical conference, FREENIX Track*, Vol. 41. California, USA, 10–5555.

[8] David Biancolin, Albert Magyar, Sagar Karandikar, Alon Amid, Borivoje Nikolić, Jonathan Bachrach, and Krste Asanović. 2021. Accessible, FPGA resource-optimized simulation of multiclock systems in firesim. *IEEE Micro* 41, 4 (2021), 58–66.

[9] Thomas Bourgeat, Clément Pit-Claudel, Adam Chlipala, and Arvind. 2020. The essence of Bluespec: a core language for rule-based hardware design. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. 243–257.

[10] Cadence. 2025. Palladium Emulation. https://www.cadence.com/en_US/home/tools/system-design-and-verification/emulation-and-prototyping/palladium.html

[11] Lu Chen, Dingyi Zhao, Zihao Yu, Ninghui Sun, and Yungang Bao. 2025. GSIM: Accelerating RTL Simulation for Large-Scale Designs. In *Proceedings of the 62nd ACM/IEEE Design Automation Conference*. 7 pages.

[12] Derek Chiou, Dam Sunwoo, Joonsoo Kim, Nikhil A Patil, William Reinhart, Darrel Eric Johnson, Jebediah Keefe, and Hari Angepat. 2007. Fpga-accelerated simulation technologies (fast): Fast, full-system, cycle-accurate simulators. In *40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2007)*. IEEE, 249–261.

[13] Deeksha Dangwal, Georgios Tzimpragos, and Timothy Sherwood. 2020. Agile hardware development and instrumentation with PyRTL. *IEEE Micro* 40, 4 (2020), 76–84.

[14] Simon Davidmann and Lee Moore. 2022. Introduction to the 5 Levels of RISC-V Processor Verification. In *Design and Verification Conference and Exhibition (DVCon)*.

[15] Subhranil Deb. 2015. *Delivering Functional Verification Engagements*. Technical Report. https://www.synopsys.com/content/dam/synopsys/services/whitepapers/delivering-functional-verification-engagements.pdf

[16] Amelia Dobis, Tjark Petersen, Hans Jakob Damsgaard, Kasper Juul Hesse Rasmussen, Enrico Tolotto, Simon Thye Andersen, Richard Lin, and Martin Schoeberl. 2021. Chiselverify: An open-source hardware verification library for chisel and scala. In *2021 IEEE Nordic Circuits and Systems Conference (NorCAS)*. IEEE, 1–7.

[17] Fares Elsabbagh, Shabnam Sheikhha, Victor A. Ying, Quan M. Nguyen, Joel S Emer, and Daniel Sanchez. 2023. Accelerating RTL Simulation with Hardware-Software Co-Design. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture* (Toronto, ON, Canada) *(MICRO '23)*. Association for Computing Machinery, New York, NY, USA, 153–166. https://doi.org/10.1145/3613424.3614257

[18] Mahyar Emami, Thomas Bourgeat, and James R. Larus. 2025. *Parendi: Thousand-Way Parallel RTL Simulation*. Association for Computing Machinery, New York, NY, USA, 783–797. https://doi.org/10.1145/3676641.3716010

[19] Mahyar Emami, Sahand Kashani, Keisuke Kamahori, Mohammad Sepehr Pourghannad, Ritik Raj, and James R. Larus. 2024. Manticore: Hardware-Accelerated RTL Simulation with Static Bulk-Synchronous Parallelism. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4* (Vancouver, BC, Canada) *(ASPLOS '23)*. Association for Computing Machinery, New York, NY, USA, 219–237. https://doi.org/10.1145/3623278.3624750

[20] Martin Erhart, Fabian Schuiki, Zachary Yedidia, Bea Healy, and Tobias Grosser. 2023. Arcilator: Fast and cycle-accurate hardware simulation in CIRCT. In *2023 LLVM Developers' Meeting*. https://llvm.org/devmtg/2023-10/slides/techtalks/Erhart-Arcilator-FastAndCycleAccurateHardwareSimulationInCIRCT.pdf

[21] Chris Fallin, Chris Wilkerson, and Onur Mutlu. 2014. The heterogeneous block architecture. In *2014 IEEE 32nd International Conference on Computer Design (ICCD)*. 386–393. https://doi.org/10.1109/ICCD.2014.6974710

[22] Harry D. Foster. 2025. *2024 Wilson Research Group IC/ASIC functional verification trend report*. Technical Report. https://resources.sw.siemens.com/en-US/white-paper-2024-wilson-research-group-ic-asic-functional-verification-trend-report/

[23] Adi Fuchs and David Wentzlaff. 2018. Scaling Datacenter Accelerators with Compute-Reuse Architectures. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. 353–366. https://doi.org/10.1109/ISCA.2018.00038

[24] Greg Gibeling, Andrew Schultz, and Krste Asanovic. 2006. The RAMP architecture & description language. In *2nd Workshop on Architecture Research using FPGA Platforms*.

[25] Ali Hajiabadi and Trevor E. Carlson. 2025. Cassandra: Efficient Enforcement of Sequential Execution for Cryptographic Programs. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 78–91. https://doi.org/10.1145/3695053.3731048

[26] Stefan Heule, Eric Schkufza, Rahul Sharma, and Alex Aiken. 2016. Stratified synthesis: automatically learning the x86-64 instruction set. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Santa Barbara, CA, USA) *(PLDI '16)*. Association for Computing Machinery, New York, NY, USA, 237–250. https://doi.org/10.1145/2908080.2908121

[27] Bo-Yuan Huang, Hongce Zhang, Pramod Subramanyan, Yakir Vizel, Aarti Gupta, and Sharad Malik. 2018. Instruction-Level Abstraction (ILA): A Uniform Specification for System-on-Chip (SoC) Verification. *ACM Trans. Des. Autom. Electron. Syst.* 24, 1, Article 10 (Dec. 2018), 24 pages. https://doi.org/10.1145/3282444

[28] Ing-Jer Huang and Alvin M Despain. 1992. High level synthesis of pipelined instruction set processors and back-end compilers. In *DAC*. Citeseer, 135–140.

[29] RISC-V Tech Hub. 2025. RISC-V Technical Specifications. https://lf-riscv.atlassian.net/wiki/spaces/HOME/pages/16154769/RISC-V+Technical+Specifications

[30] RISC-V Tech Hub. 2025. RISC-V Technical Specifications Archive. https://lf-riscv.atlassian.net/wiki/spaces/HOME/pages/16154899/RISC-V+Technical+Specifications+Archive

[31] Jaewon Hur, Suhwan Song, Dongup Kwon, Eunjin Baek, Jangwoo Kim, and Byoungyoung Lee. 2021. Difuzzrtl: Differential fuzz testing to find cpu bugs. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1286–1303.

[32] IEEE. 2020. IEEE Standard for Universal Verification Methodology Language Reference Manual. *IEEE Std 1800.2-2020 (Revision of IEEE Std 1800.2-2017)* (2020), 1–458. https://doi.org/10.1109/IEEESTD.2020.9195920

[33] Shunning Jiang, Yanghui Ou, Peitian Pan, Kaishuo Cheng, Yixiao Zhang, and Christopher Batten. 2020. PyH2: Using PyMTL3 to create productive and open-source hardware testing methodologies. *IEEE Design & Test* 38, 2 (2020), 53–61.

[34] Nursultan Kabylkas, Tommy Thorn, Shreesha Srinath, Polychronis Xekalakis, and Jose Renau. 2021. Effective processor verification with logic fuzzer enhanced co-simulation. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 667–678.

[35] Sagar Karandikar, Howard Mao, Donggyu Kim, David Biancolin, Alon Amid, Dayeol Lee, Nathan Pemberton, Emmanuel Amaro, Colin Schmidt, Aditya Chopra, et al. 2018. FireSim: FPGA-accelerated cycle-exact scale-out system simulation in the public cloud. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 29–42.

[36] Michael Katrowitz and Lisa M Noack. 1996. I'm done simulating; now what? Verification coverage analysis and correctness checking of the DEC chip 21164 Alpha microprocessor. In *Proceedings of the 33rd Annual Design Automation Conference*. 325–330.

[37] Cansu Kaynak, Boris Grot, and Babak Falsafi. 2015. Confluence: unified instruction supply for scale-out servers. In *Proceedings of the 48th International Symposium on Microarchitecture* (Waikiki, Hawaii) *(MICRO-48)*. Association for Computing Machinery, New York, NY, USA, 166–177. https://doi.org/10.1145/2830772.2830785

[38] Donggyu Kim, Christopher Celio, Sagar Karandikar, David Biancolin, Jonathan Bachrach, and Krste Asanović. 2018. DESSERT: Debugging RTL effectively with state snapshotting for error replays across trillions of cycles. In *2018 28th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 76–764.

[39] Derek Lockhart, Gary Zibrat, and Christopher Batten. 2014. PyMTL: A unified framework for vertically integrated computer architecture research. In *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE, 280–292.

[40] Albert Magyar, David Biancolin, John Koenig, Sanjit Seshia, Jonathan Bachrach, and Krste Asanović. 2019. Golden Gate: Bridging the resource-efficiency gap between ASICs and FPGA prototypes. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 1–8.

[41] Anika Malhotra and Will Chen. 2021. *Functional Chip Design Verification: When Is It Truly Finished?* Technical Report. https://www.synopsys.com/blogs/chip-

design/when-functional-chip-design-verification-truly-finished.html
[42] Albert Meixner, Michael E. Bauer, and Daniel Sorin. 2007. Argus: Low-Cost, Comprehensive Error Detection in Simple Cores. In *40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2007)*. 210–222. https://doi.org/10.1109/MICRO.2007.18
[43] Andrey Mokhov, Alexei Iliasov, Danil Sokolov, Maxim Rykunov, Alex Yakovlev, and Alexander Romanovsky. 2013. Synthesis of processor instruction sets from high-level ISA specifications. *IEEE Trans. Comput.* 63, 6 (2013), 1552–1566.
[44] SS Mukherjee, M Kontz, and SK Reinhardt. 2002. Detailed design and evaluation of redundant multi-threading alternatives. In *Proceedings 29th Annual International Symposium on Computer Architecture*. IEEE, 99–110.
[45] Anoop Mysore Nataraja. 2023. *A Research-Fertile Co-Emulation Framework for RISC-V Processor Verification.* Master's thesis.
[46] Ravi Nair and Martin E. Hopkins. 1997. Exploiting instruction level parallelism in processors by caching scheduled groups. *SIGARCH Comput. Archit. News* 25, 2 (May 1997), 13–25. https://doi.org/10.1145/384286.264125
[47] OpenXiangShan. 2025. Modern co-simulation framework for RISC-V CPUs. https://github.com/OpenXiangShan/difftest
[48] OpenXiangShan. 2025. Open-source high-performance RISC-V processor. https://github.com/OpenXiangShan/XiangShan
[49] OSCPU. 2025. RISC-V SoC designed by students in UCAS. https://github.com/OSCPU/NutShell
[50] Shruti Padmanabha, Andrew Lukefahr, Reetuparna Das, and Scott Mahlke. 2015. DynaMOS: dynamic schedule migration for heterogeneous cores. In *Proceedings of the 48th International Symposium on Microarchitecture* (Waikiki, Hawaii) *(MICRO-48)*. Association for Computing Machinery, New York, NY, USA, 322–333. https://doi.org/10.1145/2830772.2830791
[51] Shruti Padmanabha, Andrew Lukefahr, Reetuparna Das, and Scott Mahlke. 2017. Mirage cores: the illusion of many out-of-order cores using in-order hardware. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture* (Cambridge, Massachusetts) *(MICRO-50 '17)*. Association for Computing Machinery, New York, NY, USA, 745–758. https://doi.org/10.1145/3123939.3123969
[52] Nagaraju Pothineni, Philip Brisk, Paolo Ienne, Anshul Kumar, and Kolin Paul. 2010. A high-level synthesis flow for custom instruction set extensions for application-specific processors. In *2010 15th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 707–712.
[53] Hao Qian and Yangdong Deng. 2011. Accelerating RTL simulation with GPUs. In *2011 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 687–693.
[54] Shisong Qin, Chao Zhang, Kaixiang Chen, and Zheming Li. 2021. iDEV: exploring and exploiting semantic deviations in ARM instruction processing. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Virtual, Denmark) *(ISSTA 2021)*. Association for Computing Machinery, New York, NY, USA, 580–592. https://doi.org/10.1145/3460319.3464842
[55] Steven K. Reinhardt and Shubhendu S. Mukherjee. 2000. Transient fault detection via simultaneous multithreading. In *Proceedings of the 27th Annual International Symposium on Computer Architecture* (Vancouver, British Columbia, Canada) *(ISCA '00)*. Association for Computing Machinery, New York, NY, USA, 25–36. https://doi.org/10.1145/339647.339652
[56] Mehrdad Reshadi, Prabhat Mishra, and Nikil Dutt. 2003. Instruction set compiled simulation: A technique for fast and flexible instruction set simulation. In *Proceedings of the 40th Annual Design Automation Conference*. 758–763.
[57] RISC-V. 2025. Sail RISC-V model. https://github.com/riscv/sail-riscv
[58] RISC-V Software. 2025. Spike, a RISC-V ISA Simulator. https://github.com/OpenXiangShan/riscv-isa-sim
[59] riscv verification. 2024. RISC-V Verification Interface. https://github.com/riscv-verification/RVVI
[60] Benjamin John Rosser. 2018. Cocotb: a Python-based digital logic verification framework. In *Micro-electronics Section seminar. CERN, Geneva, Switzerland.*
[61] Daniel Ruelas-Petrisko, Farzam Gilani, Anoop Mysore Nataraja, Zoe Taylor, and Michael Taylor. 2025. ZynqParrot: A Scale-Down Approach to Cycle-Accurate, FPGA-Accelerated Co-Emulation. arXiv:2509.20543 [cs.AR] https://arxiv.org/abs/2509.20543
[62] Karl Rupp. 2022. 50 Years of Microprocessor Trend Data. https://github.com/karlrupp/microprocessor-trend-data
[63] Sunil Sahoo. 2024. The importance of simulation in the pursuit of absolute speed. https://blogs.sw.siemens.com/verificationhorizons/2024/04/02/the-importance-of-simulation/
[64] Sanjay Sawant and Paul Giordano. 1996. RTL emulation: The next leap in system verification. In *Proceedings of the 33rd annual Design Automation Conference*. 233–235.
[65] Timothy Sherwood, Erez Perelman, Greg Hamerly, and Brad Calder. 2002. Automatically characterizing large scale program behavior. In *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems* (San Jose, California) *(ASPLOS X)*. Association for Computing Machinery, New York, NY, USA, 45–57. https://doi.org/10.1145/605397.605403
[66] Kan Shi, Shuoxiang Xu, Yuhan Diao, David Boland, and Yungang Bao. 2023. ENCORE: Efficient Architecture Verification Framework with FPGA Acceleration. In *Proceedings of the 2023 ACM/SIGDA International Symposium on Field Programmable Gate Arrays* (Monterey, CA, USA) *(FPGA '23)*. Association for Computing Machinery, New York, NY, USA, 209–219. https://doi.org/10.1145/3543622.3573187
[67] Siemens Software. 2025. Veloce CS - IC Emulation and Prototyping. https://eda.sw.siemens.com/en-US/ic/hav/veloce-cs/
[68] Pramod Subramanyan, Bo-Yuan Huang, Yakir Vizel, Aarti Gupta, and Sharad Malik. 2017. Template-based parameterized synthesis of uniform instruction-level abstractions for SoC verification. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37, 8 (2017), 1692–1705.
[69] Pramod Subramanyan, Yakir Vizel, Sayak Ray, and Sharad Malik. 2015. Template-based synthesis of instruction-level abstractions for SoC verification. In *2015 Formal Methods in Computer-Aided Design (FMCAD)*. IEEE, 160–167.
[70] Synopsys. 2025. Emulation Systems. https://www.synopsys.com/verification/emulation-prototyping/emulation.html
[71] Zhangxi Tan, Andrew Waterman, Rimas Avizienis, Yunsup Lee, Henry Cook, David Patterson, and Krste Asanović. 2010. RAMP gold: an FPGA-based architecture simulator for multiprocessors. In *Proceedings of the 47th Design Automation Conference*. 463–468.
[72] Zhangxi Tan, Andrew Waterman, Henry Cook, Sarah Bird, Krste Asanović, and David Patterson. 2010. A case for FAME: FPGA architecture model execution. In *Proceedings of the 37th Annual International Symposium on Computer Architecture* (Saint-Malo, France) *(ISCA '10)*. Association for Computing Machinery, New York, NY, USA, 290–301. https://doi.org/10.1145/1815961.1815999
[73] Haoyuan Wang and Scott Beamer. 2023. RepCut: Superlinear Parallel RTL Simulation with Replication-Aided Partitioning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 572–585. https://doi.org/10.1145/3582016.3582034
[74] Haoyuan Wang, Thomas Nijssen, and Scott Beamer. 2025. Don't Repeat Yourself! Coarse-Grained Circuit Deduplication to Accelerate RTL Simulation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4* (Hilton La Jolla Torrey Pines, La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 79–93. https://doi.org/10.1145/3622781.3674184
[75] Kaifan Wang, Jian Chen, Yinan Xu, Zihao Yu, Zifei Zhang, Guokai Chen, Xuan Hu, Linjuan Zhang, Xi Chen, Wei He, Dan Tang, Ninghui Sun, and Yungang Bao. 2024. XiangShan: An Open-Source Project for High-Performance RISC-V Processors Meeting Industrial-Grade Standards. In *2024 IEEE Hot Chips 36 Symposium (HCS)*. 1–25.
[76] Jinyan Xu, Yiyuan Liu, Sirui He, Haoran Lin, Yajin Zhou, and Cong Wang. 2023. MorFuzz: Fuzzing processor via runtime instruction morphing enhanced synchronizable co-simulation. In *32nd USENIX Security Symposium (USENIX Security 23)*. 1307–1324.
[77] Yinan Xu, Zihao Yu, Dan Tang, Guokai Chen, Lu Chen, Lingrui Gou, Yue Jin, Qianruo Li, Xin Li, Zuojun Li, Jiawei Lin, Tong Liu, Zhigang Liu, Jiazhan Tan, Huaqiang Wang, Huizhe Wang, Kaifan Wang, Chuanqi Zhang, Fawang Zhang, Linjuan Zhang, Zifei Zhang, Yangyang Zhao, Yaoyang Zhou, Yike Zhou, Jiangrui Zou, Ye Cai, Dandan Huan, Zusong Li, Jiye Zhao, Zihao Chen, Wei He, Qiyuan Quan, Xingwu Liu, Sa Wang, Kan Shi, Ninghui Sun, and Yungang Bao. 2022. Towards Developing High Performance RISC-V Processors Using Agile Methodology. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1178–1199.
[78] Kunlin You, Yinan Xu, Kehan Feng, Luoshan Cai, Yaoyang Zhou, and Yungang Bao. 2025. *DiffTest-H: Toward Semantic-Aware Communication in Hardware-Accelerated Processor Verification.* Association for Computing Machinery, New York, NY, USA, 1462–1476. https://doi.org/10.1145/3725843.3756108
[79] Jerry Zhao, Ben Korpan, Abraham Gonzalez, and Krste Asanovic. 2020. Sonicboom: The 3rd generation berkeley out-of-order machine. In *Fourth Workshop on Computer Architecture Research with RISC-V*, Vol. 5. International Symposium on Computer Architecture Valencia, Spain, 1–7.
[80] Kexing Zhou, Yun Liang, Yibo Lin, Runsheng Wang, and Ru Huang. 2023. Khronos: Fusing Memory Access for Improved Hardware RTL Simulation. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture* (Toronto, ON, Canada) *(MICRO '23)*. Association for Computing Machinery, New York, NY, USA, 180–193. https://doi.org/10.1145/3613424.3614301
[81] Jianwen Zhu and D.D. Gajski. 2002. An ultra-fast instruction set simulator. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 10, 3 (2002), 363–373. https://doi.org/10.1109/TVLSI.2002.1043339
[82] Yan Zhu, Boru Chen, Christopher W. Fletcher, and Nandeeka Nayak. 2026. RTeAAL Sim: Using Tensor Algebra to Represent and Accelerate RTL Simulation. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (USA) *(ASPLOS '26)*. Association for Computing Machinery, New York, NY, USA, 1660–1676. https://doi.org/10.1145/3779212.3790214